

Creación de mi primer programa

Esteban Álvarez

Variables

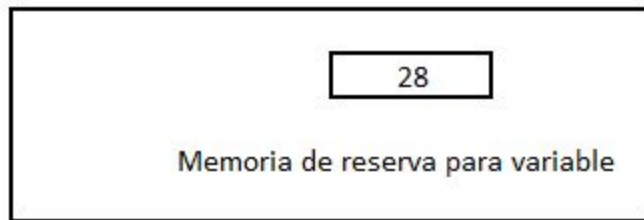
- Un programa maneja **datos** para hacer cálculos, mostrarlos, etc.
- Para trabajar con esos datos, un programa los guarda en **variables**.
- Una variable **es una zona de memoria** RAM del ordenador donde se almacena un valor.
- Las variables se definen por:
 - Un **nombre**. Permite acceder al valor que contiene la variable
 - Un **tipo de dato**. Especifica qué tipo de información almacena
 - Un **rango de valores** que puede admitir dicha variable (números, letras, etc).

tipo de datos

valor

int numero=28

nombre o
identificador



RAM

Variables

- Podemos ver una variable como **una caja**
 - A esa caja le tenemos que poner un **nombre** (con un rotulador, para saber lo que almacena)
 - No es posible tener dos cajas con el mismo nombre porque no se pueden diferenciar
 - Pero se puede tener más de una caja si tienen nombres distintos
 - La caja que creamos sólo puede almacenar un **determinado tipo** de objetos
 - Una caja creada para almacenar libros, no puede guardar revistas
 - La caja creada solo puede **almacenar UN objeto**
 - La caja de libros sólo guarda un libro
 - Se puede cambiar el libro que guarda la caja por otro

Identificadores

- Al **nombre** que damos a una variable se le llama **identificador**
- Permiten “**dirigirse**” (señalar, referenciar) a la variable creada
- Un identificador en Java:
 - Es una secuencia ilimitada sin espacios de letras y dígitos
 - No debe empezar por un número
 - Distingue entre mayúsculas y minúsculas
 - Debe ser lo más **descriptivo** posible

El identificador es el nombre que ponemos a la caja. Por ejemplo: **Libros de clase**

Identificadores

Convenciones sobre identificadores de **variables** en Java

- Se utiliza la notación **lowerCamelCase**:
 - Identificadores **SIEMPRE** en minúsculas

nombre
edad



- Si hay que utilizar **dos o más palabras**, se elimina el espacio y la siguiente inicial va en mayúsculas

nombreEmpresa
nombreAutorLibro

Identificadores

Convenciones sobre identificadores de **variables** en Java

- **Evitar eñes** (ñ), **tildes** o cualquier otro **carácter extraño**
- Aunque Java puede soportarlo, tenemos que pensar que hay teclados que no tienen determinados símbolos

Identificadores

Convenciones sobre identificadores de **constantes** en Java

- Las constantes son **variables que no cambian de valor**
- Para diferenciarlas de las variables “normales”, se utiliza la notación

UPPER_SNAKE_CASE:

- **SIEMPRE** en mayúsculas

IVA

- Si hay que utilizar **dos o más palabras**, se sustituye el espacio por un guión bajo

IVA_REDUCIDO

Identificadores

- Los identificadores deben ser **descriptivos**, es decir, deben describir **qué dato almacena** la variable
 - Identificador **correcto** para una edad:
`edad`
 - Identificador **incorrecto** para una edad:
`variable`
 - Identificador **impreciso** para una edad de jubilación:
`edad`

Más **correcto**: `edadJubilacion`

Identificadores

- Hay que **evitar las abreviaturas excesivas**

- Identificador **incorrecto** para una edad:

e

- Identificador **incorrecto** para un nombre:

n

- Aunque se recomienda utilizar **abreviaturas para palabras largas y omitir la preposición “de”**

- Identificador **correcto** para un máximo de días trabajados por una persona:

maxDiasTrabajados

- Identificador **incorrecto**

maximoDeDiasTrabajadosPorUnaPersona

enUnLugarDeLaManchaDeCuyoNombreNoQuieroAcordarme...

Identificadores

Palabras reservadas

- Las palabras reservadas (keywords) son **secuencias de caracteres cuyo uso se reserva** al lenguaje y no pueden usarse para crear identificadores
 - Busca una lista de palabras reservadas en Java
- Los IDE utilizan **colores** para diferenciar las palabras reservadas del resto del código
 - Abre Eclipse y observa de qué color se muestran las palabras reservadas, los comentarios, los errores y el resto del código

Identificadores

EJERCICIO

[Ejercicio sobre identificadores](#)

Tipos de datos

- En Java es **obligatorio** indicar el tipo de datos a la hora de declarar la variable

tipo de datos **valor**
int numero=5
nombre o
identificador

- Un tipo de datos **indica los valores que son válidos** para esa variable
 - Números, letras, etc.
- E indirectamente, las **operaciones** que se pueden realizar con ellos
 - Por ejemplo, solo se pueden sumar números

Se puede leer el libro de una caja de libros pero no la manta de una caja de mantas

Tipos de datos

Hay **dos categorías** de tipos de datos

1. Los tipos de **datos básicos o primitivos**. Son valores simples como una letra o un número
2. Tipos de datos de **referencia**. Hacen referencia a un lugar de memoria donde se almacena un grupo de valores. (Objetos y Arrays)

Tipos de datos básicos

Se clasifican en:

- Números enteros: byte, short, **int**, long
- Números reales: float, **double**
- Carácter: **char**
- Booleano o lógico: **boolean**

Tipos de datos básicos. Números enteros

- Los números enteros son los números **positivos y negativos** SIN coma (naturales)
- En Java hay **cuatro tipos de enteros**, dependiendo de lo que ocupan en la memoria RAM:
 - **byte** 1 byte. De -128 a 127
 - **short** 2 bytes. De -32.768 a 32.767
 - **int** 4 bytes. De -2.147.483.648 a 2.147.483.647
 - **long** 8 bytes. De...
- Habitualmente se usa el tipo **int**

Tipos de datos básicos. Números reales

- Los números reales son los números **positivos y negativos** CON coma
- En Java hay **dos tipos de reales**, dependiendo de lo que ocupan en la memoria RAM:
 - **float** 4 bytes
 - **double** 8 bytes
- Habitualmente se usa el tipo **double**

Tipos de datos básicos. Caracteres

- Los caracteres son **números, letras y símbolos**
- Para almacenar **UN carácter** utilizamos el tipo **char**
 - **char** 2 bytes

Tipos de datos básicos. Boolean

- El tipo de datos **booleano** almacena dos valores: **true o false** (verdadero o falso, sí o no)
 - **boolean** 1 bit
- Más adelante veremos su utilidad

Tipos de datos referenciados

- A partir de los ocho tipos de datos básicos se pueden construir los **tipos de datos referenciados**
- Es el caso de los **arrays**: un conjunto de elementos referenciados por el mismo identificador
- En este grupo también tenemos el tipo **String** (con la S mayúscula) que permite almacenar cadenas de caracteres (**palabras o frases**):

String

Declaración de variables

- La **declaración** de variables es la **creación** de las variables
- Sintaxis: se indica primero el **tipo de dato** que va a almacenar y, a continuación, el **nombre**
- Finaliza con punto y coma ;

tipoDeDato nombreVariable;

int numero;

boolean trabaja;

- Las variables se pueden declarar en cualquier bloque de código (dentro de llaves)

Declaración de variables

EJERCICIO

Declara en Eclipse las siguientes variables:

- Una variable de tipo entero y nombre “edad”
- Una variable de tipo booleano y nombre “llueve”
- Una variable de tipo real y nombre “salario”
- Una variable de tipo carácter y nombre “puerta piso”
- Una variable de tipo cadena de caracteres y nombre “raza de animal”

Declaración de variables

EJERCICIO

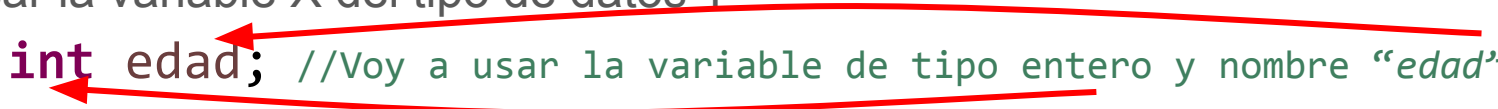
Declara en Eclipse las siguientes variables:

- Una variable de tipo entero y nombre “edad”
`int edad;`
- Una variable de tipo booleano y nombre “llueve”
`boolean llueve;`
- Una variable de tipo real y nombre “salario”
`double salario;`
- Una variable de tipo carácter y nombre “puerta piso”
`char puertaPiso;`
- Una variable de tipo cadena de caracteres y nombre “raza de animal”
`String razaAnimal;`

Declaración de variables

- Se puede ver la **declaración** como una **reserva** del tipo “en mi programa voy a usar la variable X del tipo de datos Y”

`int edad; //Voy a usar la variable de tipo entero y nombre “edad”`

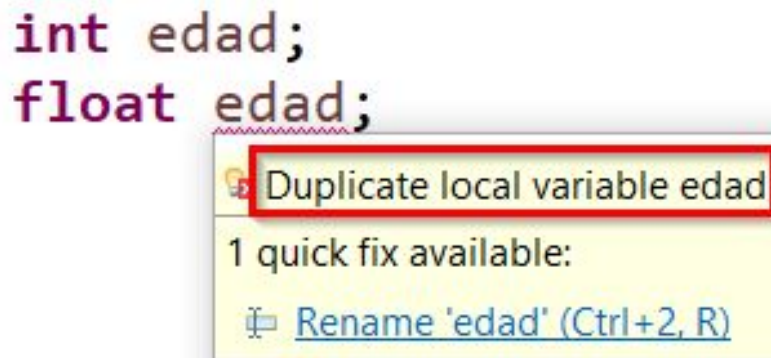


El equivalente en el ejemplo de las cajas sería: “Voy a reservar esta caja para almacenar **libros** y le voy a poner el nombre **librosDeClase**”

- A partir de este momento en `edad` sólo se puede almacenar UN número entero

Declaración de variables

- Si se declara una variable con un identificador **no se puede declarar** otra variable con el mismo nombre (aunque sea de diferente tipo)



En una llamada de teléfono:

- Hola, ¿está Antonio?
- ¿Qué Antonio, padre o hijo?

Declaración de variables

- También es posible **declarar N variables en la misma línea** siempre que sean del mismo tipo de datos

```
int edad, numeroMascotas;
```

```
String nombre, apellido1, apellido2, dni;
```

Declaración de variables

EJERCICIO

[Declaración de variables](#)

Asignación de valores

Operador de asignación (=)

- Las variables declaradas anteriormente **no tienen ningún valor**, no están inicializadas
- El operador = permite **asignar (dar) valores** a una variable ya declarada
- Se puede utilizar

- Una vez declarada la variable:

```
int numero; //Se declara la variable entera de nombre "numero"  
numero=5; //A número le asignamos el valor "5"
```



- En la propia declaración de la variable

```
int numero=5; //Se hacen dos acciones en una misma línea: se  
              //declara la variable número y se le asigna el  
              //valor 5
```



Asignación de valores

- En resumen
 - A la parte **izquierda** del = se le **asigna**
 - el **valor** que hay en la parte **derecha**

numero=5;



CUIDADO!! El operador = es el operador de asignación, no el operador “igual”

numero=5; //numero **NO** ES IGUAL a 5, a numero le asignamos el valor 5

El operador de asignación es equivalente a **meter algo en la caja**

Uso de variables

- Una vez que declaramos una variable y le asignamos un valor, el siguiente paso es **utilizarla**
- Utilizar una variable implica **acceder a su valor** para:

- **Mostrárselo** al usuario por pantalla

```
int numero=5;
```

```
System.out.println(numero); //Muestra al usuario el 5  
//Se “sustituye” número por su valor
```

- **Utilizarlo** en otras asignaciones

```
int numero1=5;
```

```
int numero2=8;
```

```
numero1=numero2; //Ahora numero1 vale lo que numero2 (8)  
//numero2 sigue valiendo lo mismo (8)
```



Uso de variables

- Una vez que declaramos una variable y le asignamos un valor, el siguiente paso es **utilizarla**
- Utilizar una variable implica **acceder a su valor** para:
 - **Utilizarla** en operaciones

```
int numero1=5;
```

```
int numero2=8;
```

```
numero1=numero2+3; //Ahora numero1 vale 11: numero2 (8) + 3 = 11
```

A red rectangular box highlights the expression 'numero2+3' in the code line 'numero1=numero2+3;'. A red arrow points from this box to the variable 'numero2' in the preceding line 'int numero2=8;'. This diagram illustrates the concept of substituting the value of a variable when it is used in an expression.

Cada vez que **usemos** una variable hay que sustituir su nombre por su valor

Es importante tener clara la **diferencia** entre asignación y uso

Uso de variables

- **IMPORTANTE:** atención a este uso:

```
int numero1=5;
```

```
int numero2=8;
```

```
numero1=numero1+numero2;
```

Uso de variables

- **IMPORTANTE:** atención a este uso:

```
int numero1=5;
```

```
int numero2=8;
```

```
numero1=numero1+numero2;
```



```
//A numero1 le asignamos el valor de numero1 (5) + el de numero2 (8)
```

```
//numero1 ahora almacena el valor 13 (5+8)
```

Uso de variables. Variables auxiliares

- **IMPORTANTE:** si una variable ya tenía un valor y se le asigna uno nuevo, **el valor antiguo se pierde**

```
int numero=5; //numero almacena el valor 5
```

```
numero=8; //Ahora almacena el valor 8. No hay forma posible  
//de acceder o recuperar el valor anterior (5)
```

- En en la caja de libros hay un ejemplar de “El Quijote”
- Quiero guardar en la caja el libro de “Los miserables”
- Para poder hacerlo, primero **destruyo** el libro de “El Quijote” y ahora ya puedo guardar el de “Los miserables”

Uso de variables. Variables auxiliares

- Si queremos guardar el valor anterior de una variable necesitamos una **variable auxiliar**

```
int numero=5;           //numero almacena el valor 5
int auxiliar=numero;    //auxiliar almacena el valor de número
numero=8;               //numero almacena el valor 8
                        //El valor anterior (5), se almacena en auxiliar
```

Para no destruir El Quijote, me hago con **otra caja** y lo guardo en ella

Uso de variables. Variables auxiliares

- El uso de variables auxiliares es la **única solución** al siguiente problema

PROBLEMA: Intercambia los valores de las variables (numero1 debe almacenar 8 y numero 2, 5)

```
int numero1=5;
```

```
int numero2=8;
```

Uso de variables. Variables auxiliares

- El uso de variables auxiliares es la **única solución** al siguiente problema

PROBLEMA: Intercambia los valores de las variables (numero1 debe almacenar 8 y numero 2, 5)

```
int numero1=5;  
int numero2=8;  
int auxiliar=numero1;//Guardo el valor de numero1 antes de modificarlo  
numero1=numero2;//numero1 vale 8, correcto. Pero numero2 también, incorrecto  
numero2=auxiliar;//Ahora numero2 vale 5. Está todo correcto
```

Si quiero intercambiar El Quijote y Los miserables, **necesito una tercera caja**

Uso de variables. Literales

- Los literales son los **valores que puede tomar una variable**
- Su conocimiento es **fundamental en la asignación** de valores a una variable
- Los literales son diferentes si el tipo de datos es
 - Entero
 - Real
 - Carácter
 - Booleano
 - String

Uso de variables. Literales de tipo entero

- Los **literales** para los tipos de datos **entero** (recuerda: byte, short, int, long) son números:
 - **enteros** (sin coma)
 - **positivos, negativos** o cero (0)
 - Para indicar un número **negativo** es obligatorio usar el símbolo -
 - El uso del símbolo + para los **positivos** es opcional y no se suele usar

El número 0 no es ni positivo ni negativo. Es 0

Uso de variables. Literales de tipo entero

- La **base decimal** es la más habitual para representar literales enteros
- Es la que usamos en nuestra vida diaria:

```
int numero1=985; //Se asigna el literal positivo 985
```

```
int numero2=-25; //Se asigna el literal negativo -25
```

```
int numero3=0; //Se asigna el literal 0
```

```
int numero4=+3; //Se asigna el literal positivo 3. Símbolo + opcional
```

Uso de variables. Literales de tipo entero

- Pero también se pueden asignar valores en **otras bases** de numeración:

- **Binario:**

- Formado por uno o más dígitos del 0 al 1
- Empiezan siempre por 0b ó 0B

```
int numeroBinario=0b0110; //Se asigna el literal binario 0110
```

- **Octal:**

- Formado por uno o más dígitos del 0 al 7
- Empiezan siempre por 0

```
int numeroOctal=071207; //Se asigna el literal octal 71207
```

- **Hexadecimal:**

- Formado por uno o más dígitos del 0 al 9 y letras de la A a la F (mayúsculas o minúsculas)
- Empiezan siempre por 0x ó 0X

```
int numeroHexa=0xF019; //Se asigna el literal hexadecimal F019
```

Uso de variables. Literales de tipo real

- Los **literales** para los tipos de datos **reales** (float y double) son números:
 - **en base decimal** (números “normales”)
 - con una **parte entera**
 - Si la parte entera es 0 puede omitirse
 - Se aplica la misma norma para positivos y negativos que en los enteros
 - un **punto decimal**
 - y una **parte fraccionaria** (decimales)

```
double numero1=-3.985;
```

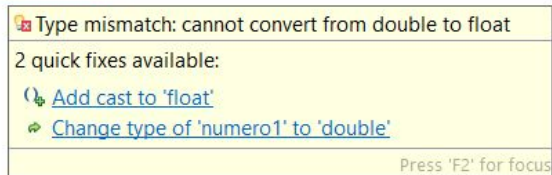
```
double numero2=.46;
```

```
double numero3=+0.985;
```

Uso de variables. Literales de tipo real

- **IMPORTANTE:** los literales de tipo real por defecto siempre son double

```
float numero1=-3.985;
```



- Si queremos que sean float hay que **añadir el sufijo f ó F**

```
float numero1=-3.985f;
```

```
float numero2=.46F;
```

Uso de variables. Literales de tipo real

EJERCICIO

[Literales de tipo numérico](#)

Uso de variables. Literales de tipo carácter

- Los **literales** de tipo de datos **char** contienen un sólo carácter encerrado entre comillas simples

```
char character1='M'; //Se asigna el literal M
```

```
char character2='j'; //Se asigna el literal j
```

```
char character3='9'; //Se asigna el literal 9
```

- Las **secuencias de escape** también son literales de tipo carácter:

```
char saltoDeLinea='\n';
```

```
char tabulador='\t';
```

- Prueba el código `System.out.println("Buenos\ndías");`
- Busca más información sobre las secuencias de escape y la tabla ASCII

Uso de variables. Literales de tipo carácter

EJERCICIO

[Literales de tipo carácter](#)

Uso de variables. Literales de tipo boolean

- El tipo boolean es **binario** por lo que sólo tiene dos literales: **true** y **false**

```
boolean llueve=false; //NO llueve
```

```
boolean encendido=true; //SÍ está encendido
```

Uso de variables. Literales de tipo String

- Están **formados** por
 - **0 o más caracteres**
 - encerrados **entre comillas dobles**
 - y pueden incluir **secuencias de escape**

```
String cadena1="Esto es una cadena de caracteres";  
String cadena2="Salto\nde línea";  
String cadena3=" "; //Cadena con un espacio en blanco
```

Uso de variables. Literales de tipo String

- Cuando queremos **inicializar un String sin valor**, tenemos dos opciones:
 - Asignarle la **cadena vacía**
`String cadena4="";` // "" es la cadena vacía
 - Asignarle el valor **null** (nada)
`String cadena5=null;` //cadena5 no almacena nada
- La **diferencia** es muy sutil pero clara:
 - `cadena4` **tiene el valor ""**
 - `cadena5` **NO tiene ningún valor** (más tarde podremos asignarle uno)

Uso de variables. Literales de tipo String

- Utilizaremos **null** para **inicializar** un String si la cadena vacía no es un valor inicial válido para esa variable String

```
String nombrePersona1="";  
String nombrePersona2=null;
```

- Pregunta: Persona1, ¿cuál es tu nombre?
- Respuesta: ""
- Pregunta: Persona2, ¿cuál es tu nombre?
- Respuesta: Todavía no tengo nombre

Uso de variables. Literales de tipo String

EJERCICIO

[Literales de tipo String](#)

EJERCICIO

Crea una tabla que resuma todos los tipos de datos (enteros, reales, carácter, booleano y cadena de caracteres) y sus literales. Utiliza esta plantilla:

TIPO	NOMBRE	FORMA DE INICIALIZAR
Enteros	byte	byte num=5;
	short	short num=-2;
	...	...
Reales		
...		

Constantes

- Ya hablamos antes sobre la **notación** de las constantes (UPPER_SNAKE_CASE)
- Aclarar que las constantes son variables cuyo **valor no se puede cambiar**
- Se **declaran, asignan y usan** igual que las variables salvo que hay que añadir la palabra reservada `final` en la declaración:

```
final double DESCUENTO=2.5;
```

```
final int MAX_ALUMNOS=25;
```

EJERCICIO

Declara una variable por cada tipo de datos que hemos visto (byte, short, float,...)

- a. Elige unos **identificadores** descriptivos para la información que almacenan
- b. Dos de estas variables son **constantes**
- c. Tres de las variables se **declaran** primero y se **inicializan** en otra línea
- d. El resto se **declaran e inicializan** en la misma línea
- e. **Muestra** por pantalla el valor de todas las variables
- f. A continuación, escribe el código para **modificar** el valor de 3 variables (no toques el código hecho en los apartados anteriores)
- g. **Vuelve a mostrar** por pantalla el valor de estas tres variables

Operadores

- Los operadores **llevan a cabo operaciones** sobre unos operandos, representados por literales (números, letras...) y/o identificadores (variables).
- Los operadores pueden ser **unarios, binarios o terciarios**, según el número de operandos que utilicen sean uno, dos o tres, respectivamente.
- Actúan sobre los tipos de **datos primitivos** y devuelven también un tipo de dato primitivo.
 - Una suma de dos números enteros devuelve un número entero
 - Una división de dos enteros devuelve un número real
 - ...

Operadores

- Los operadores se combinan con los literales y/o identificadores para formar **expresiones**.

```
numero1+3 //Esto es una expresión formada por un identificador  
          //(numero1), el operador + y el literal 3
```

- Una expresión se evalúa y produce un único resultado de un tipo de datos determinado

```
numero1+3 //Esta expresión produce un resultado que dependerá del  
          //valor de numero1
```

Operadores

- El resultado de una expresión puede se puede usar para:
 - asignárselo a una variable

```
int suma=numero1+3; //El resultado de sumar numero1+3 se asigna a
                    //la variable entera suma
```

- mostrarlo por pantalla

```
System.out.println(numero1*2); //Se muestra el resultado de multiplicar
                               //numero1 por 2
```

- `suma=numero1+3;` ya sería la acción completa por lo que tenemos una **SENTENCIA** o **INSTRUCCIÓN**
- Las instrucciones acaban siempre con punto y coma ; final

Operadores

Se pueden clasificar en las siguientes categorías:

- Aritméticos
- Relacionales
- Condicionales
- Lógicos
- De asignación

Operadores aritméticos

- Los operadores aritméticos nos permiten hacer **operaciones matemáticas**
- **Uso:**

```
variable = (valor1|variable1) operador (valor2|variable2);  
//La barra vertical | significa opcionalidad (valor1 o variable1)
```

OPERADOR	DESCRIPCION	
+	SUMA/CONCATENACION	$a + b$
-	RESTA	$a - b$
/	DIVISION	a / b
*	MULTIPLICACION	$a * b$
%	RESTO DIVISION	$a \% b$

Operadores aritméticos

- Ejemplos de uso con literales

```
suma = 3 + 7;
```

```
// Resultado: 10
```

```
resta = 5 - 2;
```

```
// Resultado: 3
```

```
multiplicacion = 3 * 2;
```

```
// Resultado: 6
```

```
division = 4 / 2;
```

```
// Resultado: 2
```

```
resto = 5 % 3;
```

```
// Resultado: 2
```

Operadores aritméticos

- Ejemplos de uso con variables

```
suma = num1+3;    // Sumamos 3 al valor de la variable num1
```

```
resta = num1-4;   // Restamos 4 al valor de la variable num1
```

EJERCICIOS

1. Declara dos variables numéricas y asígnales un valor
 - a. Calcula su suma y almacena el resultado en otra variable
 - b. Haz lo mismo para la multiplicación
 - c. Muestra la suma y la multiplicación por pantalla
 - d. Muestra por pantalla el resultado de la división y el resto pero hazlo directamente, sin guardar el resultado de las operaciones en variables

2. Haz un programa que calcule la longitud de una circunferencia según la fórmula $2 \cdot \pi \cdot r$
 - a. π debe ser una constante con una precisión de 4 decimales
 - b. r es una variable que almacena el radio. Dale un valor cualquiera de tipo real
 - c. Crea otra variable para el resultado
 - d. Muestra el resultado por pantalla

EJERCICIOS

3. Haz un programa que calcule el descuento de un producto
 - a. Utiliza una constante de tipo entero para el porcentaje de rebaja con el valor que quieras de 0 a 100
 - b. Crea una variable con el precio del producto. Asígnale el valor que quieras
 - c. Crea otra variable donde se almacena el descuento que se aplica al artículo según la fórmula: $\text{descuento} = \text{precio} * \text{porcentaje de rebaja} / 100$
 - d. Muestra el resultado por pantalla

4. [Más ejercicios de operadores aritméticos](#)

Operadores aritméticos

- Hay operaciones matemáticas para las que **no existe** un operador aritmético
- **Potencia**
 - Para elevar un número `n` a una potencia `a` se usa `Math.pow(n, a)` ;
`double n=Math.pow(23,2); // 23 al cuadrado`
`double n=Math.pow(2,3); // 2 al cubo`
- **Raíz cuadrada**
 - Para la raíz cuadrada de `n` se usa `Math.sqrt(n)` ;
`double n=Math.sqrt(4); // Raíz cuadrada de 4`
`double n=Math.sqrt(328); // Raíz cuadrada de 328`

Operadores aritméticos

EJERCICIOS

Determinar el tipo del resultado en operaciones aritméticas

Operadores aritméticos unarios

- Son operadores que solo requieren **un parámetro** para funcionar

Operador	Descripción
+	Operador unario suma. Indica un número positivo
-	Operador unario resta. Invierte el signo de un número
++	Operador de incremento. Incrementa el valor en 1
--	Operador de decremento. Decrementa el valor en 1
!	Operador de complemento lógico. Invierte el valor de un booleano

Operadores aritméticos unarios

Operadores unarios de suma o resta

- El uso del **operador unitario suma** es redundante
- El operador unitario **resta** **invierte el signo** de un número:

```
int num1=2;  
int num2=-7;  
System.out.println(-num1); //Imprimirá por pantalla un -2  
num1=-num2;                //num1 almacena 7 (positivo)
```

Operadores aritméticos unarios

Operadores de incremento y decremento

- La variable a la que se aplica estos operadores **incrementa/decrementa** su valor en 1:

```
++ variable; //Equivale a: variable=variable+1
```

```
variable ++; //Equivale a: variable=variable+1
```

```
-- variable; //Equivale a: variable=variable-1
```

```
variable --; //Equivale a: variable=variable-1
```

Operadores aritméticos unarios

Operadores de incremento y decremento

- Hay que tener cuidado en su uso como **prefijo** o como **sufijo** en una asignación
- Uso como **prefijo**

```
suma=++variable;
```

```
//Es lo mismo que:
```

```
variable=variable+1;
```

```
suma=variable;
```

Operadores aritméticos unarios

Operadores de incremento y decremento

- Hay que tener cuidado en su uso como **prefijo** o como **sufijo** en una asignación
- Uso como **sufijo**

```
suma=variable++;
```

```
//Es lo mismo que:
```

```
suma=variable;
```

```
variable=variable+1;
```

EJERCICIO

- Crea una variable entera `numero1` con el valor 5
- Crea una variable entera `numero2` con el valor que contiene `numero1`. No se permite hacer `numero2=5`
- `numero1` y `numero2` valen lo mismo. Ejecuta estas líneas de código:

```
System.out.println(numero1++);  
System.out.println(++numero2);
```
- Analiza por qué el resultado es diferente
- ¿Qué valor tienen las dos variables? Compruébalo con:

```
System.out.println(numero1);  
System.out.println(numero2);
```

Operadores aritméticos unarios

EJERCICIO

[Operadores de incremento y decremento](#)

Operadores aritméticos unarios

Operador de complemento lógico

- **Cambia el valor** de una expresión o variable booleana (false->>true; true->>false)

```
boolean llueve=false;  
System.out.println(llueve); //Muestra false  
System.out.println(!llueve); //Muestra true
```

EJERCICIOS

Hacer las Actividades iniciales del Campus Virtual

Operadores relacionales

- Se utilizan para **comparar** datos de tipo primitivo: numérico, carácter y booleano
- El resultado se utilizará en otras expresiones o sentencias que se ejecutarán dependiendo de si se cumple o no la relación
- Siempre dan como **resultado** un valor booleano: true o false

Operadores relacionales

Operador	Descripción
==	igual a
!=	no igual a (distinto)
>	mayor que
>=	mayor o igual que
<	menor que
<=	menor o igual que

Operadores relacionales

- Ejemplos de uso:

```
num1=5;
```

```
num2=3;
```

```
boolean res=num1>4;
```

```
res=num1<=num2;
```

```
System.out.println(num2==num1);
```

```
System.out.println(14!=2);
```

Operadores relacionales

- Ejemplos de uso:

```
num1=5;
```

```
num2=3;
```

```
boolean res=num1>4; //res almacena true
```

```
res=num1<=num2; //res almacena false
```

```
System.out.println(num2==num1); //Muestra false por pantalla
```

```
System.out.println(14!=2); //Muestra true por pantalla
```

Operadores relacionales

Ejercicios

Operadores lógicos

- Los operadores lógicos realizan operaciones sobre valores booleanos, o resultados de expresiones relacionales
- Dan como resultado un valor booleano

Operador	Descripción
&&	Operador AND (Y)
	Operador OR (O)

Operadores lógicos. AND

- El operador AND es **muy exclusivo**
- Sólo devuelve **true** cuando los dos operandos son true

El arcoiris sale cuando **llueve Y hace sol**. Se tienen que cumplir las dos condiciones a la vez, si falla una, no sale el arcoiris:

```
llueve=false;
haceSol=true;
boolean arcoiris=llueve && haceSol;    //arcoiris=false. No llueve

llueve=true;
haceSol=true;
boolean arcoiris=llueve && haceSol;    //arcoiris=true
```

Operadores lógicos. AND

- Este operador se suele utilizar con los **operadores relacionales** vistos anteriormente

EJERCICIO:

int a=4;

int b=6;

- ¿True or False?
 - $(a < 5) \ \&\& \ (b > 5)$
 - $(a < 6) \ \&\& \ (b > 6)$
 - $(a < 6) \ \&\& \ (b \geq 6)$
 - $(a \neq 4) \ \&\& \ (b > 0)$
 - $(a == 4) \ \&\& \ (b \neq 0)$

Expresión A	Expresión B	A && B
Verdadero	Verdadero	Verdadero
Verdadero	Falso	Falso
Falso	Verdadero	Falso
Falso	Falso	Falso

Operadores lógicos. OR

- El operador OR es **más permisivo**
- Devuelve **true** cuando **UNO** de los dos operandos es true

Utilizamos botas de agua cuando **llueve** **O** el **suelo está mojado**.
Cuando se cumpla **una de las dos** condiciones, nos ponemos botas.
Solo **NO** nos ponemos botas si ni llueve ni el suelo está mojado

```
llueve=false;
sueloMojado=true;
boolean botas=llueve || sueloMojado; //Suelo mojado. Me pongo botas

llueve=false;
sueloMojado=false;
boolean botas=llueve || sueloMojado; //No me pongo botas. botas=false
```

Operadores lógicos. OR

- Este operador se suele utilizar con los **operadores relacionales** vistos anteriormente

EJERCICIO:

int a=4;

int b=6;

- ¿True or False?
 - $(a < 5) \parallel (b > 5)$
 - $(a < 6) \parallel (b > 6)$
 - $(a < 6) \parallel (b \geq 6)$
 - $(a \neq 4) \parallel (b < 0)$
 - $(a == 4) \parallel (b \neq 0)$

Expresión A	Expresión B	A && B
Verdadero	Verdadero	Verdadero
Verdadero	Falso	Verdadero
Falso	Verdadero	Verdadero
Falso	Falso	Falso

EJERCICIO

Si X, Y y Z son variables de tipo boolean con valores X = true, Y = false, Z = true, determina el valor de las siguientes expresiones lógicas:

1. $(X \ \&\& \ Y) \ || \ (X \ \&\& \ Z)$
2. $(X \ || \ !Y) \ \&\& \ (!X \ || \ Z)$
3. $X \ || \ (Y \ \&\& \ Z)$
4. $!((X \ || \ Y) \ \&\& \ Z)$
5. $(X \ || \ Y) \ || \ ((X \ \&\& \ !Z) \ \&\& \ !Y)$
6. $!X \ || \ ((!Y \ || \ Z) \ \&\& \ (X \ \&\& \ !Y))$
7. [Más ejercicios](#)

Operadores de asignación

- Ya hemos visto el operador de asignación =
- Pero existen otros operadores de asignación **compuestos** que asignan y a la vez hacen una **operación aritmética**

Operador	Ejemplo	Forma extendida
+=	a+=5	a=a+5
-=	a-=b	a=a-b
=	a=b	a=a*b
/=	a/=2	a=a/2
%=	a%=b	a=a%b

Operadores de asignación

- El **uso** de operadores compuestos es típico cuando:
 - Queremos **sumar** (o cualquier otra operación) un valor a una variable **y guardar el resultado en la misma variable**

```
int ingresos=200;
```

```
ingresos+=50; //ingresos se incrementa en 50
```

- Igual que lo anterior pero **el valor está en otra variable**

```
int ingresos=200;
```

```
int paga=138;
```

```
ingresos+=paga; //ingresos se incrementa en 138
```

Combinación de operadores

- Los operadores **se pueden combinar** en una misma instrucción para obtener un resultado
- **Combinaciones habituales:**
 - Operadores **aritméticos**

- Combinamos varios operadores aritméticos para hacer operaciones matemáticas complejas:

//Ejemplo de cálculo con operadores aritméticos

//Devuelve un número real que almacenamos

```
double res=ingresos-(2*gastos)/50;
```

//Ejemplo que añade el operador +=

```
res+=ingresos-(2*gastos)/50;
```

Combinación de operadores

- Los operadores **se pueden combinar** en una misma instrucción para obtener un resultado
- **Combinaciones habituales:**
 - Operadores **lógicos y relacionales**
 - Ya se han visto ejemplos
 - El resultado de evaluar la expresión devuelve un booleano indicando si se cumplen o no las restricciones

```
//Ejemplo de combinación de operadores lógicos y relac.  
//Devuelve un booleano que indica si se cumple la condición  
boolean aprobado=(nota>=5)&&(nota<=10);
```

Combinación de operadores

- Los operadores **se pueden combinar** en una misma instrucción para obtener un resultado
- **Combinaciones habituales:**
 - Operadores **lógicos y relacionales** con operadores **matemáticos**
 - Es una extensión del ejemplo anterior
 - El resultado de evaluar la expresión devuelve un booleano indicando si se cumplen o no las restricciones
 - Estas restricciones añaden operaciones matemáticas

```
//Ejemplo de combinación de todos los operadores  
//Devuelve un booleano que indica si se cumple la condición  
boolean aprobado=(examen*0.8+tarea*0.2>=5) && !(faltas>6);
```

EJERCICIO

Dadas las variables de tipo entero con valores $A = 5$, $B = 3$, $C = -12$ indicar si la evaluación de estas expresiones da como resultado verdadero o falso:

a) $A > 3$

b) $A > C$

c) $A < C$

d) $B < C$

e) $B \neq C$

f) $A == 3$

g) $A * B == 15$

h) $A * B == -30$

i) $C / B < A$

j) $C / B == -10$

k) $C / B == -4$

l) $A + B + C == 5$

m) $(A+B == 8) \ \&\& \ (A-B == 2)$

n) $(A+B == 8) \ || \ (A-B == 6)$

o) $A > 3 \ \&\& \ B > 3 \ \&\& \ C < 3$

p) $A > 3 \ \&\& \ B \geq 3 \ \&\& \ C < -3$

Operadores de Strings. Comparación

- Para saber si **dos strings son iguales** no se puede usar el operador ==
- Necesitamos **utilizar** `equals`

```
String cadena1="Hola";
```

```
String cadena2="Hola";
```

```
boolean iguales=cadena1==cadena2; //devuelve siempre false
```

```
boolean iguales=cadena1.equals(cadena2); //true
```

```
boolean iguales=cadena2.equals(cadena1); //true
```

```
boolean iguales=cadena1.equals("Hola"); //true
```

```
boolean iguales=cadena1.equals("hola"); //false
```

```
boolean iguales=cadena1.equalsIgnoreCase("hola"); //true
```

Operadores de Strings. Concatenación

- Para **concatenar** (unir) dos o más cadenas se usa el operador +

```
String c1="Hola";
```

```
String c2="Juan";
```

```
System.out.println(c1+c2); //Muestra: HolaJuan
```

```
System.out.println(c1+c2+"."); //Muestra: HolaJuan.
```

```
String saludo=c1+", qué tal"+c2+"."; //Espacio para separar
```

El nombre de una variable va sin "" si queremos concatenar su valor

```
int numero=5;
```

```
System.out.println("numero="+numero); //numero=5
```

Literales

- Llegados a este punto, hay que recalcar la **importancia de los literales**
- Revisa el siguiente código:

```
num1=5;
```

```
num2=5.0;
```

```
num3='5';
```

```
num4="5";
```

- ¿Almacenan la **misma información**?

Literales

- Almacenan la **misma información**: el valor 5
- Pero de diferente manera

```
num1=5;    //Almacena 5 en entero
```

```
num2=5.0;  //Almacena 5 en real
```

```
num3='5';  //Almacena 5 en carácter
```

```
num4="5";  //Almacena 5 en String
```

Literales

Cuidado con la declaración de las variables y los literales!

```
int    num1= 5;    //Los literales enteros van sin comillas
double num2= 5.0;  //Literales reales: con .
char   num3= '5';  //Literales char con comillas simples
String num4= "5";  //Literales String con comillas dobles
```

Literales

¿Es correcta la siguiente comparación?

```
int num1= 5;
```

```
boolean mayor=num1>"5";
```

Literales

¿Es correcta la siguiente comparación?

```
int num1= 5;
```

```
boolean mayor=num1>"5";
```



No es correcto. Estamos comparando num1 que almacena un 5 entero con un 5 String. No se pueden comparar porque son tipos de datos diferentes

Cuando usamos una variable (en comparaciones, asignaciones, operaciones aritméticas, etc.) hay que utilizar **literales** del mismo tipo de datos

Conversión de tipos de datos. Casting

- Cualquier dato puede ser convertido a otro tipo de datos
- Ejemplo:

```
int numero1=7;  
double numero2=numero1;
```

- En el ejemplo anterior se está haciendo **cast** (conversión de tipos): el valor entero 7 almacenado en numero1 se almacena en numero2 que es de tipo double
- Al hacer esa asignación 7 pasa de ser 7 entero a 7 real (double)
- Muestra por pantalla numero1 y numero2 y observa la diferencia

Conversión de tipos de datos. Cast implícito

```
int numero1=7;  
double numero2=numero1;
```

- En la asignación `numero2=numero1` se ha hecho un **cast implícito** (conversión implícita o automática)
- Se produce un **cast automático o implícito** cuando:
 - Los tipos de datos son compatibles
 - Asignamos un tipo de datos pequeño a un tipo de datos más grande
- Jerarquía en las conversiones implícitas:

byte -> short -> int -> long -> float -> double

Conversión de tipos de datos. Cast implícito

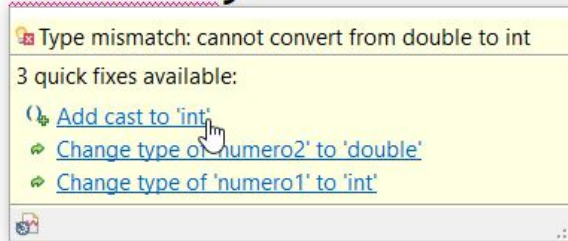
```
int numero='A';
```

- Aquí se está produciendo un **cast implícito**: numero almacena el valor 65 que es el código asociado a A en la tabla ASCII

Conversión de tipos de datos. Cast explícito

- Sin embargo, si asignamos un double a una variable entera se produce el error **type mismatch** (tipos incompatibles)

```
double numero1=7.3;  
int numero2=numero1;
```



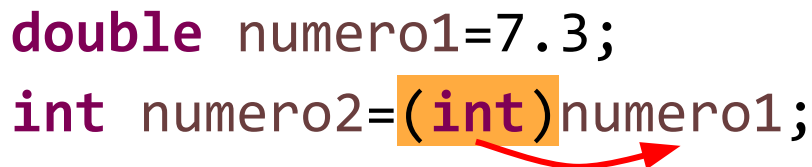
- En el propio mensaje ya aparece una solución: **hacer cast a int**

Type mismatch es uno de los **errores más típicos** cuando empezamos a programar

Conversión de tipos de datos. Cast explícito

- Si pulsamos sobre la solución de **hacer cast** obtenemos la expresión:

```
double numero1=7.3;  
int  numero2=(int)numero1;
```



- Ese **(int)** a la izquierda de una variable o un literal le aplica una **conversión** al tipo de datos entero

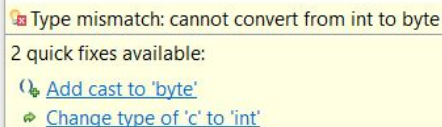
A veces el cast puede implicar una **pérdida de información**

- Muestra por pantalla numero2 y observa si devuelve 7.3

Conversión de tipos de datos. Cast explícito

- El error **type mismatch** también se suele producir en las **expresiones**:

```
byte a = 23;  
byte b = 34;  
byte c = a + b;
```



Type mismatch: cannot convert from int to byte
2 quick fixes available:
[Add cast to 'byte'](#)
[Change type of 'c' to 'int'](#)

- La suma de dos números **devuelve un int** aunque estos números estén almacenados en formato byte

```
byte c = (byte) (a + b);
```

Conversión de tipos de datos. Cast explícito

- Otras veces no hay errores de compilación pero la **ejecución no es correcta**

```
double division=7/2;
```

```
System.out.println(division); //Muestra 3.0
```

- 7/2 devuelve 3 (entero) porque 7 y 2 son **literales enteros** y por tanto, se hace la **división entera**
- Luego al asignar 3 a division que es float, se hace un **cast implícito** y pasa a valer 3.0
- Solución: hacer que uno de los dos literales **sea double**

```
double division=(double)7/2; //ATENCIÓN!! Es un cast de 7 a double
```

```
double division=7/2.0; double division=7f/2;
```

Precedencia de operadores

- ¿Qué **resultado** devuelve el cálculo: $2 + 3 \times 4$? ¿20?
- El resultado **no es 20** porque la multiplicación tiene precedencia sobre la suma y se aplica $2 + (3 \times 4)$
- La tabla de precedencias de operadores es compleja por eso es importante **tener en cuenta** las siguientes precedencias:
 - La multiplicación y la división tienen precedencia sobre la suma y la resta
 - El operador Y (&&) tiene precedencia sobre el O (||)

Precedencia de operadores

- **Recomendación:** en caso de duda, usar paréntesis
 - $2 + 3 \times 4 = 14$
 - Pero si no estás seguro, añade paréntesis: $2 + (3 \times 4) = 14$
 - Aunque podemos modificar la precedencia con: $(2 + 3) \times 4 = 20$

Esto no es nuevo, en **matemáticas** ya te has encontrado con este problema

EJERCICIO

Escribe en Java las siguientes expresiones algebraicas:

a.
$$x = \frac{a + \frac{b}{c}}{a} + d$$

b.
$$x = \frac{a + \frac{b}{c}}{\frac{a}{b} + c}$$

c.
$$x = \frac{b^2 - 4ac}{2a}$$

d. [Más expresiones](#)

e. [Expresiones algorítmicas](#)

Comentarios en Java

- Se trata de **texto que introduce el programador para explicar** bloques de código, apoyando la documentación y haciendo más legible y mantenible la aplicación
- Este texto no se tiene en cuenta por el compilador: **es como si no estuviese allí**
- Pueden ser
 - de una sola línea
 - de múltiples líneas o de bloque
 - de documentación

Comentarios en Java

Comentario de una sola línea

- Se utiliza el delimitador `//` y a continuación se añade el texto

```
//Comentario de una sola línea
```

Para comentar varias líneas en Eclipse, seleccionalas y utiliza el atajo **Ctrl+7**

Comentarios en Java

Comentario de múltiples líneas o de bloque

- Se utiliza el delimitador `/*` al principio del párrafo y se cierra con `*/`

```
/*El comentario comienza  
continúa  
y continúa  
El comentario termina*/
```

Comentarios en Java

Comentario de documentación o Javadoc

- Permiten generar documentación automáticamente por medio de herramientas externas
- Se utiliza el delimitador `/**` al principio del párrafo y se cierra con `*/`
- Se recomienda utilizar `*` al principio de cada línea

```
/**  
 * Este método se encarga de iniciar la ejecución del programa  
 * Éste es el método principal del proyecto  
*/
```

FIN!